Intelligent Image Document Analyzer

PREPARED BY

Jawad Ali

Table of Contents

1. Introduction 5
- 1.1 Project overview and purpose 5
- 1.2 Problem statement 5
- 1.3 Project objectives 5
- 1.4 Scope and main features 5
- 1.5 Expected input and output 5
- 1.6 About this report 6

2. Requirements and Technology Stack 6
- 2.1 Requirements 6
- 2.2 Technology stack 6
- 2.3 Tesseract OCR is an external requirement 7

3. System Architecture and Workflow 7
- 3.1 Overall design 7
- 3.2 End-to-end invoice processing 9
- 3.3 Module interaction 11
- 3.4 API request-to-response flow 12

4. Project Directory Structure 13
- 4.1 Actual structure 13
- 4.2 Purpose of each item 13

5. Module-by-Module Implementation 14
- 5.1 app/main.py (API layer) 14
- 5.2 app/pipeline.py (coordinator) 15
- 5.3 app/image_processing.py (preprocessing) 16
- 5.4 app/ocr.py (Tesseract integration) 16
- 5.5 app/text_cleaning.py 17
- 5.6 app/extraction.py 17
- 5.7 app/schemas.py 17
- 5.8 Other files that belong to the application 18
- 5.9 How the modules work together 18

6. Invoice Extraction Logic 18
- 6.1 General rules for labelled fields 18
- 6.2 Field by field 19
- 6.3 Amount normalization 19
- 6.4 Vendor name 20
- 6.5 Missing-field behavior and fallback 20
- 6.6 Illustrative example of the fallback 20
- 6.7 Known limits of the extraction logic 21

7. API Documentation 21
- 7.1 GET / 21
- 7.2 GET /health 21
- 7.3 POST /upload 21
- 7.4 Using Swagger UI 23
- 7.5 Application Demonstration and API Output 23

8. Automated Testing 26
8.1 Types of tests used in this project 26
8.2 Test files 27
8.3 Commands 29
8.4 Test results 29
9. OCR Evaluation Experiments 29
9.1 What scripts/evaluate_ocr.py is 29
9.2 What it varies 29
9.3 How to run it 30
9.4 How to read the output 30
9.5 Experiments, tests and benchmarks are different things 30
10. Installation and Execution Guide 31
10.1 Prerequisites 31
10.2 Clone the repository 31
10.3 Create the environment and install dependencies 31
10.4 Install and verify Tesseract OCR 31
10.5 Optional quick check 32
10.6 Start the server 32
10.7 Open Swagger UI and upload an invoice 32
10.8 Run the tests 32
10.9 Run OCR evaluation experiments 32
10.10 Common problems 32
11. Input Validation and Error Handling 33
11.1 Safeguards implemented 33
11.2 Pipeline exceptions 35
11.3 Purpose and limits of these controls 35
12. Results and Limitations 36
12.1 Implemented capabilities 36
12.2 Limitations 36
13. Future Improvements 37
14. Conclusion 37
15. References and Appendices 37
15.1 References 37
15.2 Appendix A: Files generated for this documentation 38
15.3 Appendix B: Items not verified 38
15.4 Appendix C: Glossary 38

List of Figures and Tables

Figures

Figure 1: System architecture. Solid arrows are the normal flow. Dotted arrows are optional or error paths. Source: diagrams/01-system-architecture.mmd. 8
Figure 2: End-to-end invoice processing flowchart, including validation failures and the fallback passes. Source: diagrams/02-invoice-processing-flowchart.mmd. 10
Figure 3: Which module uses which. Solid arrows are imports used at run time. Dotted arrows show the evaluation script, which is run by hand. Source: diagrams/03-module-interaction.mmd. 11
Figure 4: Sequence of calls for a successful upload. Source: diagrams/04-api-sequence.mmd. 12
Figure 5: Swagger UI of the Intelligent Image Document Analyzer, showing the API version and the endpoints GET /, GET /health and POST /upload. 24
Figure 6: The POST /upload form in Swagger UI, with the required file field and the Execute button. 24
Figure 7: A successful POST /upload response (HTTP 200) shown in Swagger UI. 25
Figure 8: How the test files relate to the modules and the test data. Dotted lines connect each test file to the module it tests. Source: diagrams/05-testing-structure.mmd. 28
Figure 9: Order of the upload checks and the HTTP status returned for each failure. Source: diagrams/06-error-handling.mmd. The "Pillow" checks are in validate_image in app/main.py. 34

Tables

Table 1: Project requirements 6
Table 2: Technology stack and the role of each package 6
Table 3: Purpose of each file and directory 13
Table 4: Constants in app/main.py 14
Table 5: Routes defined in app/main.py 14
Table 6: Error mapping in POST /upload 15
Table 7: Public functions of app/image_processing.py 16
Table 8: Exception types in app/ocr.py 16
Table 9: Main internal parts of app/extraction.py 17
Table 10: Labels and value rules for each field 19
Table 11: Examples of amount normalization 19
Table 12: API endpoints 21
Table 13: Request parameter of POST /upload 22
Table 14: Response fields of POST /upload 22
Table 15: Status codes of POST /upload 22
Table 16: Error responses and their messages 23
Table 17: Types of tests used in the project 26
Table 18: Test files, purpose and important scenarios 27
Table 19: Variations in Experiment 1 29
Table 20: Variations in Experiment 2 30
Table 21: Manual experiment, automated test and formal benchmark compared 30
Table 22: Common problems and fixes 32
Table 23: Input validation and error handling safeguards 35
Table 24: Pipeline exceptions and how the API handles them 35
Table 25: Implemented capabilities and the tests that cover them 36
Table 26: Known limitations 36
Table 27: Proposed improvements (not implemented) 37
Table 28: References 37
Table 29: Documentation files 38
Table 30: Items that could not be verified 38
Table 31: Glossary 38

1. Introduction

1.1 Project overview and purpose

The Intelligent Image Document Analyzer is a Python web service. A user uploads a photo or scan of an invoice. The service reads the text in the image with OCR (Optical Character Recognition) and returns the text together with six extracted invoice fields as JSON.

The project is built as a REST API with FastAPI. It uses OpenCV to prepare the image, Tesseract to read the text, and plain Python rules to find the fields. It does not use machine learning for field extraction.

1.2 Problem statement

Invoices are often received as images. The information in them (invoice number, date, vendor and amounts) must be typed into other systems by hand, which is slow and easy to get wrong. Computers cannot use the text inside an image until it is converted to text. OCR output is also noisy: it can contain broken labels, extra symbols, and values in the wrong order. A useful system must therefore read the image, clean the text, and find the fields without trusting noisy text blindly.

1.3 Project objectives

1. Accept an invoice image through an HTTP API and reject unsafe or invalid files.
2. Prepare the image so Tesseract can read it better.
3. Convert the image to text and normalize the text.
4. Extract six invoice fields with clear, testable rules.
5. Return `null` for a field instead of a guess when the value is not reliable.
6. Recover missing fields with extra OCR passes, without replacing values that were already found.
7. Keep the code split into small modules, each covered by automated tests.

1.4 Scope and main features

In scope (implemented):

- Image upload with validation of size, format, pixel count and file content.
- Preprocessing: grayscale, noise reduction and thresholding.
- OCR with Tesseract through `pytesseract`.
- Text cleaning (whitespace and line endings only).
- Rule-based extraction of: invoice number, invoice date, vendor name, subtotal, tax amount, total amount.
- An optional second and third OCR pass that fill missing fields (enabled by the API).
- A JSON response with image details, raw OCR text, cleaned text and the six fields.
- Swagger UI at `/docs` (generated automatically by FastAPI).
- A script for comparing OCR settings by eye, and an automated test suite.

Out of scope (not implemented): user accounts or authentication, a database, saving uploaded files, PDF input, line-item extraction, languages other than English (`eng` is the default and the pipeline does not change it), machine-learning or LLM-based extraction, and a graphical front end.

1.5 Expected input and output

Input: one image file sent as `multipart/form-data` in a form field named `file`. Allowed formats are JPEG, PNG, WebP and TIFF. The maximum size is 10 MB and the maximum image size is 40 million pixels.

Output: a JSON object. The example below shows the shape only. Every value in it is an invented placeholder, not the result of a real run:

```
{
  "filename": "example_invoice.png",
  "format": "PNG",
  "width": 1000,
  "height": 1400,
  "size_bytes": 123456,
  "status": "processed",
  "message": "...",
  "raw_ocr_text": "...",
  "cleaned_text": "...",
  "invoice_fields": {
    "invoice_number": "INV-0001",
    "invoice_date": "2026-01-15",
    "vendor_name": "Example Supplies Ltd",
    "subtotal": "100.00",
    "tax_amount": "10.00",
    "total_amount": "110.00"
  }
}
```

Amounts are decimal strings. The invoice number and date are returned as written in the OCR text. A field that cannot be found is `null`.

1.6 About this report

This report describes the design and implementation of the project in the GitHub repository `jawadali-s/intelligent-document-analyzer`. It does not report new test runs, speed measurements, coverage figures or accuracy figures, because none are claimed. Details that could not be confirmed are marked **[Not verified]** and listed in section 15.3.

2. Requirements and Technology Stack

2.1 Requirements

Table 1: Project requirements

Requirement	Details
Operating system	Ubuntu Linux commands are used in this report. Other systems are not covered.
Python	Python 3 with `venv` and `pip`. The repository does not state a minimum version.
Python packages	Listed with exact versions in `requirements.txt` (section 2.2).
Tesseract OCR	External program. It is not installed by `pip` and must be installed separately (section 2.3).
Git	Needed to clone the repository.

2.2 Technology stack

Table 2: Technology stack and the role of each package

Technology	Version in `requirements.txt`	Actual role in this project
Python	Not pinned in the repository.	Language of the whole project.
FastAPI	0.143.0	Web framework. Defines the routes, reads the upload, produces the Swagger UI and the OpenAPI schema.
Uvicorn	0.54.0	Server that runs the FastAPI app (`python -m uvicorn app.main:app`).
python-multipart	0.0.32	Lets FastAPI read `multipart/form-data` file uploads. Required for `UploadFile`.
Pydantic	2.14.1	Defines and validates the response models in `app/schemas.py`.

Technology	Version in `requirements.txt`	Actual role in this project
Pillow	12.3.0	Validates uploads in `app/main.py`: detects the real format, checks the pixel count, and detects corrupt or truncated images.
OpenCV (`opencv-python`)	5.0.0.93	Decodes image bytes (`cv2.imdecode`), converts to grayscale, blurs, thresholds and resizes.
NumPy	2.5.4	Images are NumPy arrays throughout the processing code.
Tesseract OCR	External program, not installed by pip.	The OCR engine that turns an image into text.
pytesseract	0.3.13	Python wrapper that calls the `tesseract` program.
pytest	9.1.1	Test runner for all automated tests.
Other pinned packages	see `requirements.txt`	Support libraries installed with the above. The file looks like a full `pip freeze`, so it also lists indirect dependencies.

2.3 Tesseract OCR is an external requirement

Tesseract is an external requirement. `pip install` does not install it. It must be installed on the operating system (on Ubuntu: `sudo apt install tesseract-ocr`) and the `tesseract` command must be on the `PATH`. If it is missing, the code raises `OCREngineNotFoundError` and the API returns a 500 response.

3. System Architecture and Workflow

3.1 Overall design

The application has two layers.

- **API layer** (`app/main.py`, `app/schemas.py`). It receives the upload, validates it, calls the pipeline, converts errors to HTTP status codes and builds the response.
- **Processing layer** (`app/pipeline.py` and the modules it calls). It turns image bytes into text and fields. It does not know about FastAPI or HTTP.

Each processing step is its own module with one public entry function. `app/pipeline.py` only coordinates them and contains no image, OCR, cleaning or extraction logic of its own. This makes every step easy to test alone with fakes.

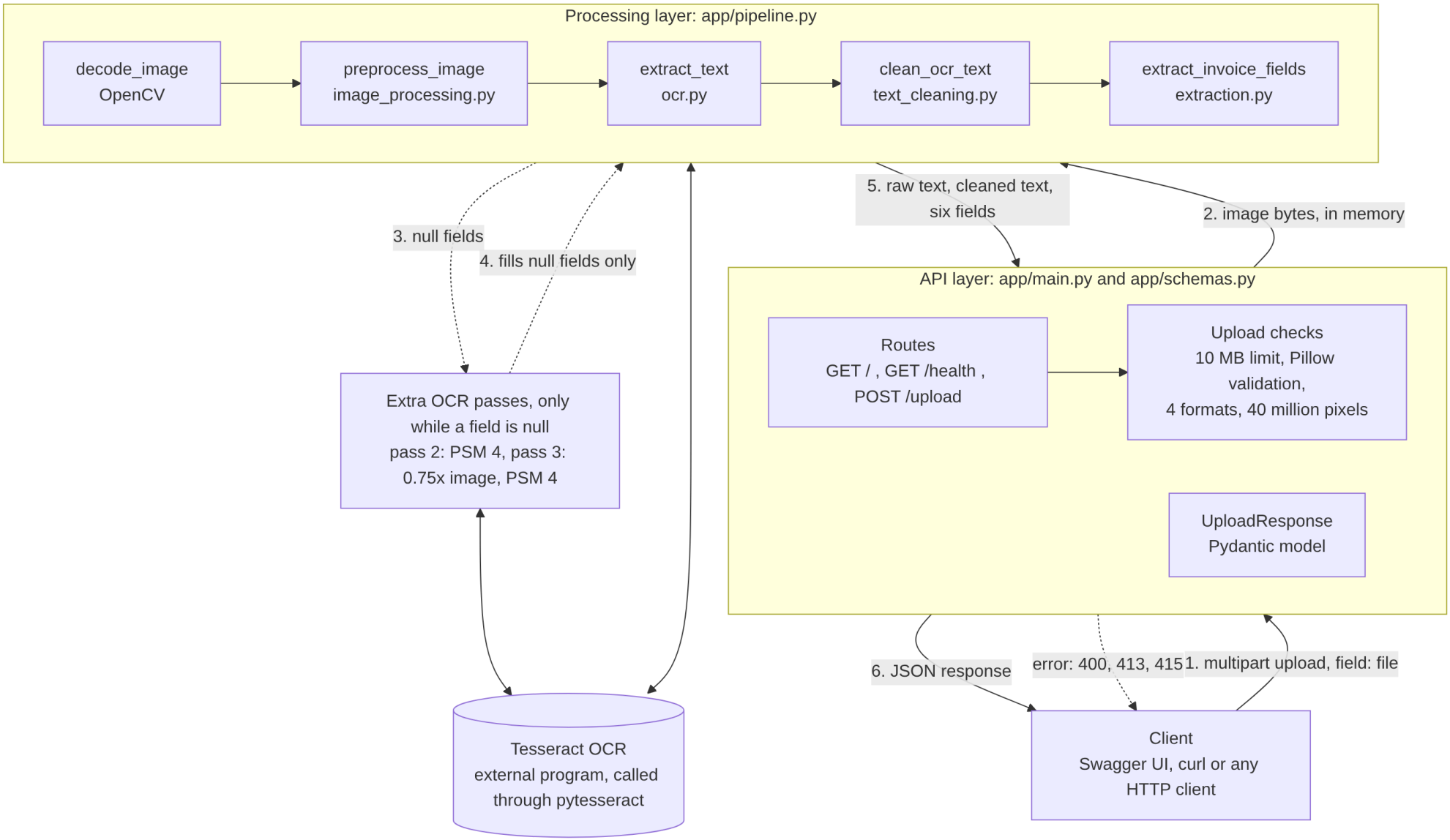


Figure 1: System architecture. Solid arrows are the normal flow. Dotted arrows are optional or error paths. Source: `diagrams/01-system-architecture.mmd`.

3.2 End-to-end invoice processing

The steps for one `POST /upload` request are:

1. **Upload and size check.** The file is read in 1 MB chunks into memory. Over 10 MB gives 413. An empty file gives 400.
2. **Image validation (Pillow).** Format, pixel count and file integrity are checked. Failures give 400, 413 or 415.
3. **Decode (OpenCV).** The bytes become a BGR image array. A failure gives 422.
4. **Preprocess.** Grayscale, Gaussian blur (kernel 3) and Otsu threshold give a black-and-white image.
5. **OCR pass 1.** Tesseract reads the image with page segmentation mode (PSM) 6. A Tesseract error gives 500.
6. **Clean text.** Whitespace and line endings are normalized.
7. **Extract fields.** Rules find the six fields. Unreliable fields stay `null`.
8. **Fallback processing (only if a field is `null`).** The API enables two extra passes: - Pass 2: the same preprocessed image is read again with PSM 4. Only `null` fields are filled. - Pass 3: if a field is still `null`, a copy of the image resized to 0.75x is read with PSM 4. Only `null` fields are filled.
9. **Response.** The `UploadResponse` is returned. `raw_ocr_text` and `cleaned_text` always come from pass 1, even if later passes filled fields.

If pass 2 or pass 3 fails with an OCR error, the error is logged as a warning and the earlier result is kept. The request does not fail.

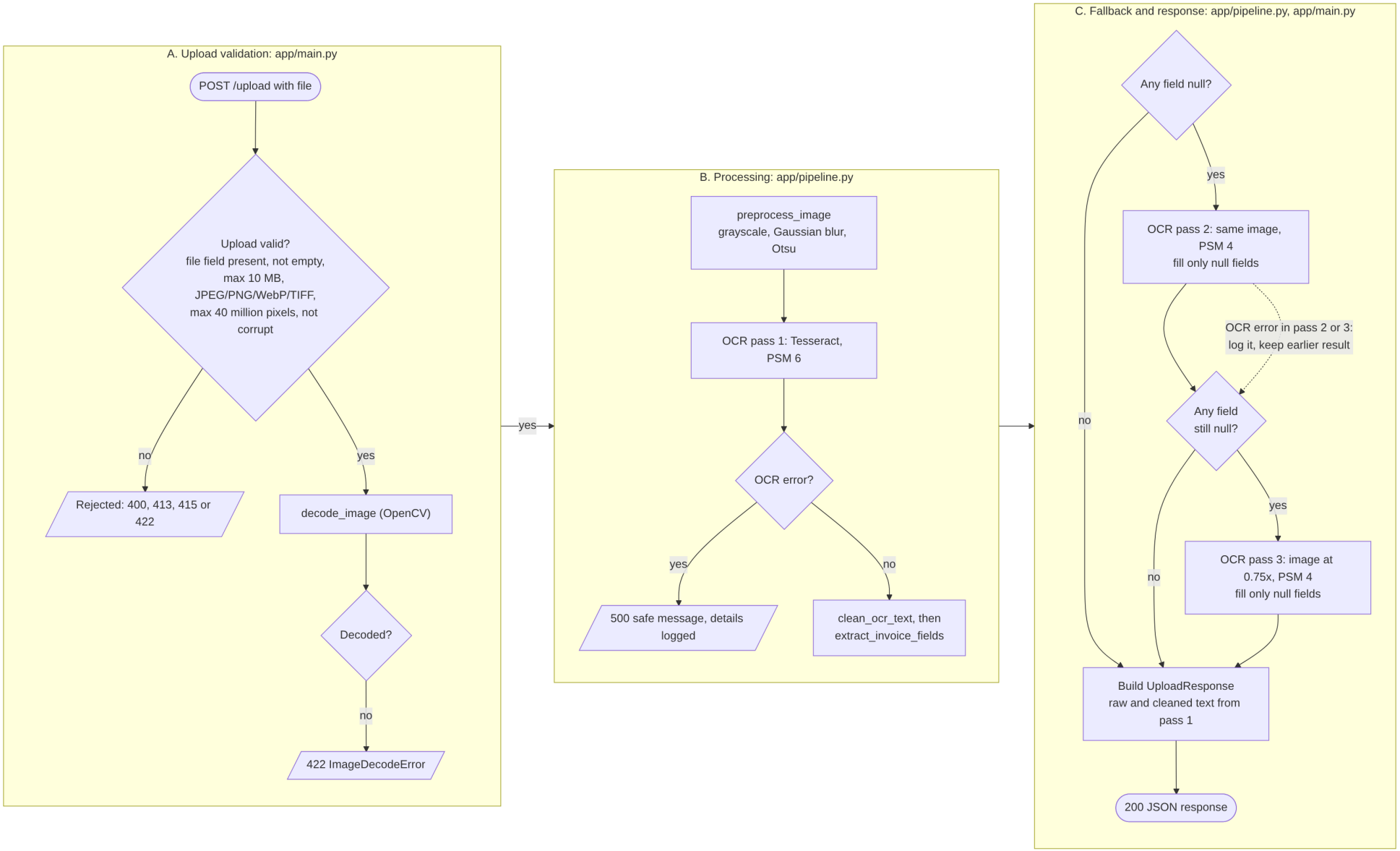


Figure 2: End-to-end invoice processing flowchart, including validation failures and the fallback passes. Source: `diagrams/02-invoice-processing-flowchart.mmd`.

3.3 Module interaction

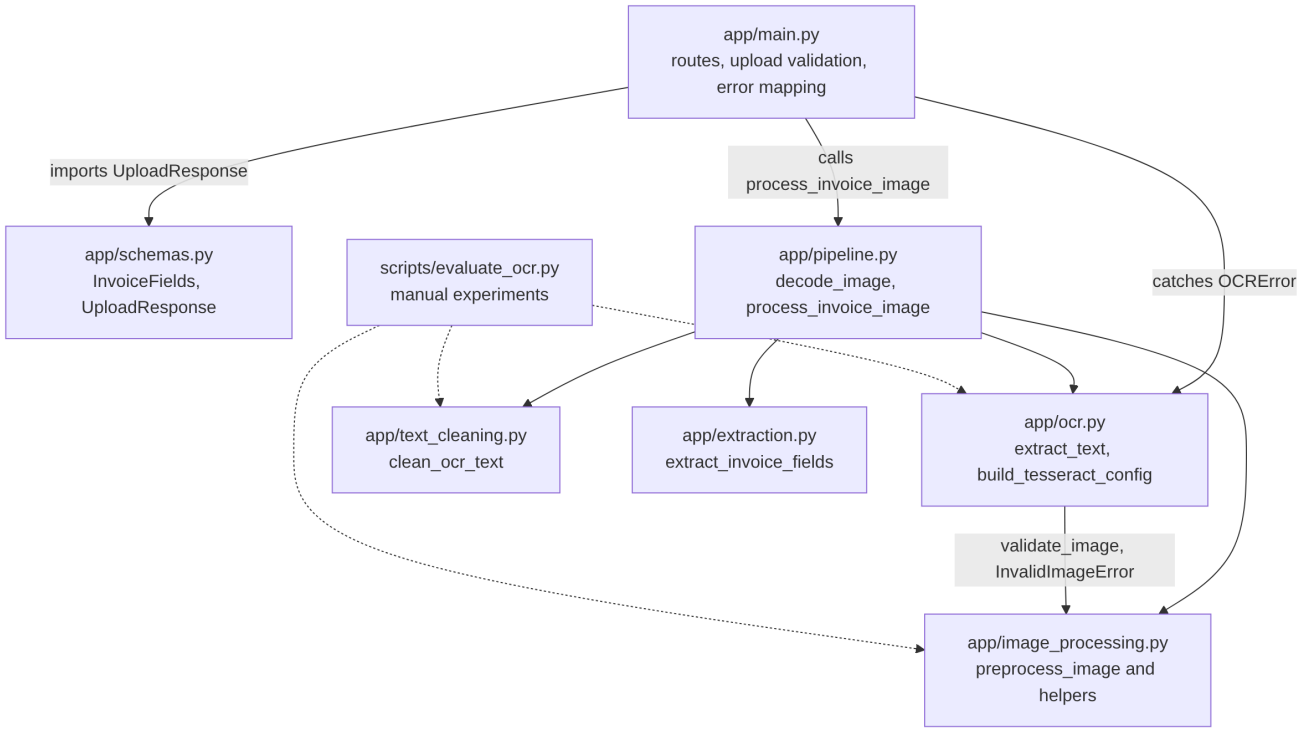


Figure 3: Which module uses which. Solid arrows are imports used at run time. Dotted arrows show the evaluation script, which is run by hand. Source: `diagrams/03-module-interaction.mmd`.

Notes on Figure 3:

- `app/main.py` imports `OCRError` from `app/ocr.py` only to translate it into a 500 response. It has its own `validate_image` function for uploads and does not use the one in `app/image_processing.py`.
- `app/ocr.py` imports `validate_image` and `InvalidImageError` from `app/image_processing.py` to check the array it is given.
- `app/text_cleaning.py`, `app/extraction.py` and `app/schemas.py` do not import other project modules.

3.4 API request-to-response flow

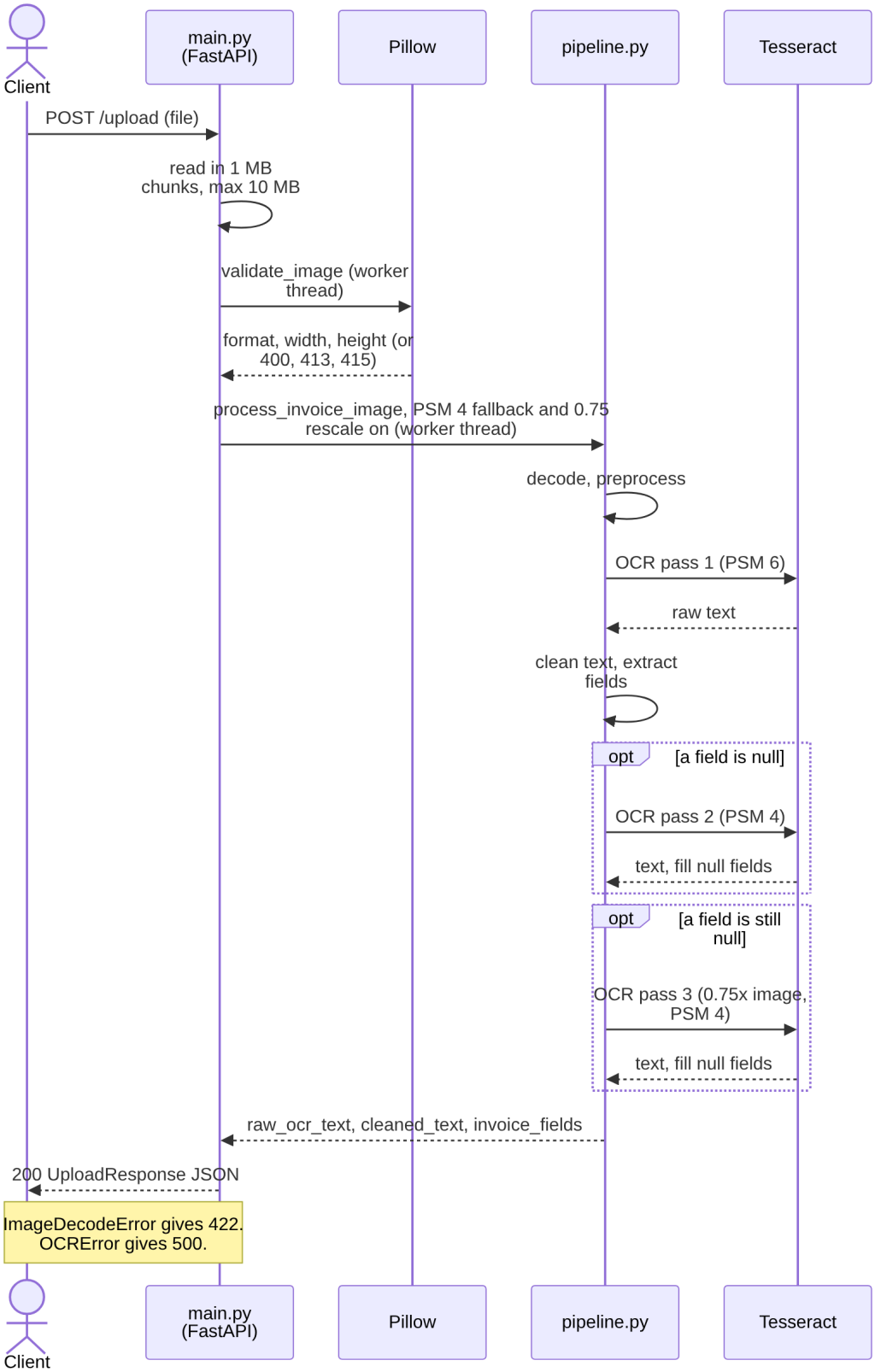


Figure 4: Sequence of calls for a successful upload. Source: `diagrams/04-api-sequence.mmd`.

Two blocking steps run outside the event loop with `run_in_threadpool`: `validate_image` and `process_invoice_image`. This keeps the server able to answer other requests while OCR is running. A test (`test_pipeline_runs_in_a_worker_thread_not_in_the_event_loop`) checks this.

4. Project Directory Structure

4.1 Actual structure

This is the structure of the project source, with `__pycache__` omitted. Files generated for this documentation are not part of the application and are listed separately in Appendix A.

```
intelligent-document-analyzer/
├── app/
│   ├── __init__.py             (empty)
│   ├── main.py
│   ├── pipeline.py
│   ├── image_processing.py
│   ├── ocr.py
│   ├── text_cleaning.py
│   ├── extraction.py
│   └── schemas.py
├── samples/
│   └── invoice_testing_dataset/
│       └── invoice_test_01.png ... invoice_test_09.png   (9 images)
├── scripts/
│   └── evaluate_ocr.py
├── tests/
│   ├── invoice_ocr_samples.py
│   ├── invoice_regression_cases.py
│   ├── test_api.py
│   ├── test_extraction.py
│   ├── test_image_processing.py
│   ├── test_invoice_regression.py
│   ├── test_ocr.py
│   ├── test_pipeline.py
│   └── test_text_cleaning.py
├── .gitignore
├── README.md
└── requirements.txt
```

4.2 Purpose of each item

Table 3: Purpose of each file and directory

Path	Purpose
`app/main.py`	FastAPI application, constants and limits, upload validation, the three routes, error-to-status mapping.
`app/pipeline.py`	Coordinates decode, preprocess, OCR, clean and extract. Implements the fallback passes.
`app/image_processing.py`	OpenCV preprocessing: grayscale, noise reduction, thresholding.
`app/ocr.py`	Tesseract integration and OCR error types.
`app/text_cleaning.py`	OCR text cleaning: line endings, whitespace and blank lines.
`app/extraction.py`	Rule-based extraction of the six invoice fields.
`app/schemas.py`	Pydantic response models.
`app/__init__.py`	Empty file that makes `app` a package.
`samples/invoice_testing_dataset/`	Sample invoice images (`invoice_test_01.png` to `invoice_test_09.png`).
`scripts/evaluate_ocr.py`	Manual experiment script that prints OCR output for different settings. It does not score results.
`tests/test_*.py`	Automated tests, one file per module plus a regression file. They are described in chapter 8.
`tests/invoice_ocr_samples.py`	Test data only: recorded OCR text (PSM 6 and PSM 4) and expected fields for three invoices.
`tests/invoice_regression_cases.py`	Test data only: ten `Case` records used by the regression tests.

Path	Purpose
`requirements.txt`	Pinned Python dependencies.
`.gitignore`	Ignores `.venv/`, Python caches, test artifacts (`.pytest_cache/`, `.coverage`, `htmlcov/`), `.env`, `.vscode/` and `.DS_Store`.
`README.md`	Project overview, setup and usage for general readers.

Configuration. The project has no configuration files such as `pyproject.toml`, `pytest.ini`, `.env` or a Dockerfile. All limits are constants in `app/main.py` and `app/ocr.py`. `.gitignore` mentions `.env`, but the application code does not read environment variables. The only environment variable used is `RUN_OCR_INTEGRATION`, read by one test.

5. Module-by-Module Implementation

5.1 `app/main.py` (API layer)

Purpose. Creates the FastAPI app (`title="Intelligent Image Document Analyzer"`, `version="0.3.0"`), validates uploads and exposes the routes.

Constants

Table 4: Constants in `app/main.py`

Constant	Value	Meaning
`MAX_UPLOAD_SIZE_BYTES`	10 × 1024 × 1024	Maximum upload size.
`READ_CHUNK_SIZE_BYTES`	1024 × 1024	Size of each read from the upload.
`MAX_IMAGE_PIXELS`	40,000,000	Maximum width × height. Also set as `Image.MAX_IMAGE_PIXELS` for Pillow.
`ALLOWED_IMAGE_FORMATS`	JPEG, PNG, WEBP, TIFF	Accepted formats (detected from content).
`FALLBACK_PSM`	4	PSM for pass 2 and pass 3.
`RESCUE_SCALE`	0.75	Resize factor for pass 3.

Important functions

- `read_upload_with_limit(upload, destination) -> int`. Rejects the upload early if `upload.size` is known and too large. Otherwise reads 1 MB chunks into a `BytesIO`, stops with 413 as soon as the running total passes 10 MB, and returns the size.
- `validate_image(image_stream) -> (format, width, height)`. Opens the image with Pillow in two passes. Pass one checks the format and pixel count, then calls `verify()`. Pass two reopens the image, checks again, and calls `load()` to decode all pixels, which catches truncated files. A `warnings.catch_warnings()` block turns Pillow's decompression-bomb warning into an error only inside this function.
- `upload_invoice_image(file)`. The `POST /upload` handler. It is `async`, calls `process_invoice_image` through `run_in_threadpool`, and always closes the buffer and the upload in a `finally` block.

Routes

Table 5: Routes defined in `app/main.py`

Method and path	Returns
`GET /`	`{"project": "Intelligent Image Document Analyzer", "message": "API is running"}`
`GET /health`	`{"status": "healthy"}`
`POST /upload`	`UploadResponse` (see section 5.7).

Error mapping

Table 6: Error mapping in `POST /upload`

Condition	Status	Detail
Missing `file` field	422	FastAPI default validation error.
Empty file	400	`Uploaded file is empty.`
Not an image, or corrupt/incomplete	400	`File is not a recognizable image.` / `Image is corrupt or incomplete.`
Over 10 MB	413	`File is too large. Maximum allowed size is 10 MB.`
Too many pixels (including decompression bomb)	413	`Image dimensions are too large.`
Format not allowed	415	`Unsupported image format: ...`
`ImageDecodeError` from the pipeline	422	Fixed message `DECODE_FAILURE_MESSAGE`.
`OCRError` from the pipeline	500	Fixed message `OCR_FAILURE_MESSAGE`. The real error is logged with `logger.error`, not returned.

Any other exception is not caught, so a programming error is not hidden as a 4xx response (a test checks this).

Design decisions. Nothing is written to disk. The format is detected from content, not from the filename or content type. The OCR error message sent to the client is fixed so that internal paths or Tesseract output are not exposed.

5.2 `app/pipeline.py` (coordinator)

Purpose. Runs the processing steps in order and applies the optional fallback passes.

Public items

- `decode_image(image_bytes) -> ndarray`. Uses `cv2.imdecode(..., cv2.IMREAD_COLOR)`. Returns a `uint8` array of shape (H, W, 3) in BGR order. Raises `InvalidImageBytesInputError` (a `TypeError`) for a wrong input type and `ImageDecodeError` (a `ValueError`) for empty or undecodable data.
- `process_invoice_image(image_bytes, *, fallback_psm=None, rescue_scale=None) -> dict`. Returns a dictionary with `raw_ocr_text`, `cleaned_text` and `invoice_fields`.

Internal helpers

- `_fill_missing_fields(...)`. Runs one extra OCR pass and copies a value into the result only where the current value is `None`. It logs how many fields were filled and their names, never their values.
- `_rescale(image, scale)`. Resizes with `INTER_AREA` when shrinking and `INTER_CUBIC` when enlarging.
- `_validate_rescue_scale(...)`. Accepts a number above 0 and up to 4.0, other than 1.

Fallback rules

- Both options are off by default (`None`). The API turns them on.
- `fallback_psm` is checked first with `build_tesseract_config`, so a bad mode fails with `OCRConfigError` before any OCR runs.
- `rescue_scale` needs `fallback_psm`. Otherwise a `ValueError` is raised before OCR runs.
- A value found earlier is never replaced by a later pass.

The core of the logic (shortened):

```
invoice_fields = extract_invoice_fields(cleaned_text)

if fallback_psm is not None and any(v is None for v in invoice_fields.values()):
    invoice_fields = _fill_missing_fields(invoice_fields, preprocessed, fallback_psm)
    if rescue_scale is not None and any(v is None for v in invoice_fields.values()):
        invoice_fields = _fill_missing_fields(
            invoice_fields, _rescale(preprocessed, rescue_scale), fallback_psm, ...
        )
```

Third pass. Pass 3 reads a copy of the preprocessed image resized by `rescue_scale` (0.75 in the API) with the fallback PSM. Like pass 2, it fills only missing fields. Extra passes are not guaranteed to improve a result.

5.3 `app/image_processing.py` (preprocessing)

Purpose. Prepares an image for OCR using OpenCV. Images are `uint8` NumPy arrays.

Public functions

Table 7: Public functions of `app/image_processing.py`

Function	What it does
`preprocess_image(image, *, color_order="BGR", denoise_method="gaussian", kernel_size=3, threshold_method="otsu", block_size=31, c=10.0)`	Runs the three steps below and returns a 2-D array containing only 0 and 255. The input is never modified.
`to_grayscale(image, color_order="BGR")`	Converts 1-, 3- or 4-channel input to one channel.
`reduce_noise(gray, method, kernel_size)`	Gaussian or median blur. A kernel size of 1 means no blur.
`threshold_otsu(gray)`	Global Otsu threshold.
`threshold_adaptive(gray, block_size, c)`	Local threshold. Not used by the default pipeline.
`validate_image(image)`	Checks that an input is a valid image array. Reused by `ocr.py`.

The pipeline calls `preprocess_image(image)` with defaults, so the actual processing is: grayscale, Gaussian blur with kernel 3, Otsu threshold.

Exceptions. `ImagePreprocessingError` (a `ValueError`) is the base class. `InvalidImageError` is raised for a bad image array. `InvalidParameterError` is raised for an unsupported option such as an unknown threshold method or an even kernel size.

5.4 `app/ocr.py` (Tesseract integration)

Purpose. Runs Tesseract on an image array and returns the text exactly as Tesseract produced it.

Public items

- `extract_text(image, language="eng", psm=6, extra_config=None) -> str`.
- `build_tesseract_config(psm=6, extra_config=None) -> str`. Builds strings such as `--psm 6`. It rejects `--psm` and `-l` inside `extra_config`.
- `validate_language(language)`. Accepts codes such as `eng` or `eng+fra`.
- `SUPPORTED_PSM_VALUES`: 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13. Modes 0 and 2 do not return text and are rejected.

Exceptions

Table 8: Exception types in `app/ocr.py`

Exception	Raised when
`OCRError`	Base class of all below.

Exception	Raised when
`OCRInputError` (also `ValueError`)	The image array is invalid.
`OCRConfigError` (also `ValueError`)	Language, PSM or extra config is invalid.
`OCREngineError` (also `RuntimeError`)	Tesseract failed.
`OCREngineNotFoundError`	The `tesseract` program is not found. The message explains how to install it.
`OCRLanguageDataError`	The language data cannot be loaded.

Design decision. All Tesseract failures are converted to one `OCRError` family. The API only needs to catch `OCRError`.

5.5 `app/text_cleaning.py`

Purpose. Normalizes line endings and whitespace in OCR text.

`clean_ocr_text(text) -> str` does only these things:

- `\r\n` and `\r` become `\n`.
- Runs of whitespace inside a line become one space, and each line is trimmed.
- Several blank lines become one blank line.
- Leading and trailing blank lines are removed.

Line breaks are kept, because the extractor needs to know when a label is alone on a line and its value is on the next one. A non-string input raises `InvalidTextInputError` (a `TypeError`).

5.6 `app/extraction.py`

Purpose. Finds the six fields in cleaned text with rules. The rules are explained in section 6.

Public items: `extract_invoice_fields(text) -> dict[str, str | None]`, `FIELD_NAMES` and `InvalidExtractionInputError` (a `TypeError` raised for non-string input).

Result. Always a new dictionary with exactly six keys, in this order: `invoice_number`, `invoice_date`, `vendor_name`, `subtotal`, `tax_amount`, `total_amount`. Empty text gives six `None` values.

Main internal parts

Table 9: Main internal parts of `app/extraction.py`

Part	Role
Label pattern tuples (`_INVOICE_NUMBER_LABELS`, `_DATE_LABELS`, `_SUBTOTAL_LABELS`, `_TAX_LABELS`, `_TOTAL_LABELS`)	Ordered lists of label patterns, most specific first.
`_FieldSpec` and `_FIELD_SPECS`	Describe, per field, its labels, its value parser and its search options.
`_parse_invoice_number`, `_parse_date`, `_parse_amount`, `_parse_printed_amount`	Value parsers. Each returns the value or `None`.
`_extract_strict_field`, `_extract_relaxed_field`, `_extract_from_label_rows`	The three label-search stages.
`_find_vendor_name`, `_find_vendor_from_label`	Vendor logic.

The module does not use any external library for matching beyond Python's `re`, `datetime.date` and `decimal.Decimal`.

5.7 `app/schemas.py`

Purpose. Defines the response shape so FastAPI validates it and documents it in Swagger.

- `InvoiceFields`: the six fields, each `str | None`. All six are required, so every response contains all six keys, with `null` for missing ones.

- `UploadResponse`: `filename`, `format`, `width`, `height`, `size_bytes`, `status`, `message`, `raw_ocr_text`, `cleaned_text` and `invoice_fields`. `status` is typed `Literal["processed"]`, so it always equals `"processed"` in a successful response. Errors are returned by raising `HTTPException`, not through this model.

Amounts are strings, not floats, to avoid rounding problems.

5.8 Other files that belong to the application

- `app/__init__.py`: empty.
- `scripts/evaluate_ocr.py`: a command-line experiment script (not part of the API). Usage: `python scripts/evaluate_ocr.py IMAGE [--experiment 1|2|all]`. Experiment 1 compares default preprocessing against no blur (`kernel_size=1`) with PSM 4, 6 and 11. Experiment 2 compares scales 1x, 2x and 3x, grayscale versus Otsu, with PSM 4 and 6. It prints raw and cleaned text for manual comparison. It does not score, rank or write files. Details are in chapter 9.

5.9 How the modules work together

1. `main.py` validates the upload and passes the raw bytes to `pipeline.process_invoice_image`.
2. `pipeline.py` calls `decode_image`, then `image_processing.preprocess_image`, then `ocr.extract_text`, then `text_cleaning.clean_ocr_text`, then `extraction.extract_invoice_fields`.
3. If fields are missing, `pipeline.py` calls `ocr.extract_text` again (with a different PSM and scale) and reuses `text_cleaning` and `extraction` on the new text.
4. `main.py` puts the result into `schemas.UploadResponse`.

Modules pass plain data (bytes, arrays, strings, dictionaries) to each other. The main exception types connect the layers: `ImageDecodeError` and `OCRError` travel up from the pipeline and are converted to HTTP errors in `main.py`.

6. Invoice Extraction Logic

6.1 General rules for labelled fields

These rules apply to invoice number, date, subtotal, tax and total.

1. **Labels are matched at the start of a line, ignoring upper or lower case.** A label must not run into a longer word, so `Total` matches but `Totals` does not. This also means `Subtotal` is never read as `Total`, and `Due Date` is never read as `Date`.
2. **The value is the rest of the line**, after an optional `:` or `|`. If the label is alone on its line, the value is the next non-empty line.
3. **Labels are tried in priority order.** For each label, lines are tried from the top. The first valid value wins. An invalid value is skipped and the search continues.
4. **No fuzzy matching.** A corrupted label such as `ice No` (from `Invoice No`) is not repaired. If no valid label is found, the field is `null`.
5. **No calculation.** A missing subtotal, tax or total is never computed from the other amounts.

If rule 1 to 3 give no value, two more stages run, in this order:

- **Relaxed search.** Looks for labels after stray punctuation (`; Subtotal 100.00`), at the start of a table cell (`... | Total | 110.00`), and, for specific labels only, in the middle of a line (`... Invoice no. INV-1`). The value must still be the whole rest of the line, ignoring stray punctuation tokens such as a trailing `.`. In the middle of a line the bare labels `Date`, `Tax` and `Total` are never used, and amounts must have two decimals. One exception: `Tax` followed by a rate, for example `Tax (5%) PKR 11,400.00`.
- **Label-row reading.** Some invoices print a row of labels and then a row of values (`INVOICE NUMBER ISSUE DATE DUE DATE`, then `AB-100 2026-01-15 2026-02-14`). The values are matched to labels by position. This is used only if

every cell in the label row is a known label and the number of values fits. `Due Date`, `Terms` and PO cells are recognized but never used.

6.2 Field by field

Table 10: Labels and value rules for each field

Field	Labels, in priority order	Value rules
`invoice_number`	`Invoice No` / `Invoice Number` / `Invoice #`; `Bill No` / `Bill Number` / `Bill #`; `Inv No` / `Inv #`	Returned exactly as written, so leading zeros are kept. Allowed characters: letters, digits, `-`, `_`, `/`, `.`. Must contain a digit and be at most 40 characters. A `#` before the value is dropped.
`invoice_date`	`Invoice Date`, `Issue Date`, `Bill Date`, `Date`	Returned exactly as written, not converted, after a check that it is a real calendar date. Accepted forms: `2026-09-15`, `14/10/2026`, `14-10-2026`, `24 October 2026`, `12-Oct-2026`, `Oct 03, 2026`, `19.10.2026`. For `DD/MM/YYYY` the string is accepted if day-first or month-first is a real date. Dotted dates are read day-first.
`subtotal`	`Subtotal` (also `Sub-total`, `Sub total`), `Net Amount`, `Taxable Amount`	Amount normalization (section 6.3).
`tax_amount`	`Sales Tax`, `Tax Amount`, `Total Tax`, `VAT`, `GST`, `Tax`	Amount normalization. A rate may follow the label: `Tax 17%`, `VAT (17%)`, `GST (18%)`, `GST at 18%`, `GST @ 18%`. The rate is skipped, not returned.
`total_amount`	`Grand Total`, `Total Due` / `Total Amount Due`, `Amount Due`, `Total Amount`, `Total Payable`, `Total`. Only if none gives a value: `Balance Due`.	Amount normalization. `Balance Due` is last because a balance can be lower than the total after payments.
`vendor_name`	Explicit `FROM:` label, otherwise position (section 6.4)	Text of the vendor line.

For dates, the first three labels may also take the next line when their own line holds no valid date. This handles side-by-side layouts where the label shares a line with an unrelated column and the date is alone on the next line. The bare `Date` label does not do this.

6.3 Amount normalization

The result is a decimal string with no currency symbol and no thousands separator. Decimal places are kept as written, so `2,500.00` becomes `2500.00`.

Table 11: Examples of amount normalization

Written in OCR text	Result	Rule
`Rs. 2,500.00`, `$1,350.00`, `2500 PKR`	`2500.00`, `1350.00`, `2500`	A currency symbol or code (`Rs`, `PKR`, `USD`, `EUR`, `GBP`, `INR`, `AED`, `SAR`, `$`, `€`, `£`, `₹`) may come before or after the number.
`1,234,567.00`	`1234567.00`	International grouping.
`12,34,567.00`	`1234567.00`	Indian grouping.
`5.190,00`, `986,10`	`5190.00`, `986.10`	European format. Accepted **only** with a decimal comma followed by exactly two digits. `6.176` and `100.000` are rejected because they could be either style. Mixed forms such as `1.250,000.00` or `12,50.00` are rejected.
`%2,646.36`	`2646.36`	A `%` directly before a printed amount with two decimals is read as an OCR misread of a currency symbol. A rate such as `18%` is never an amount. Other unknown symbols are not guessed.

Amounts are handled as `decimal.Decimal` internally and formatted back to a string. If the text is not a valid amount, the result is `None`.

6.4 Vendor name

1. **Explicit label.** A line starting with `From:` (the colon is required, so `From Scratch Bakery` is not a label). The value is the text after the colon. A line that also mentions `Bill to`, `Billed to`, `Ship to` or `Sold to` is rejected.
2. **First suitable line.** Otherwise the first non-empty line that is not a generic title, not a decorative line (no letters or digits, such as `-----`) and not a table row (a `|` before the first letter or digit). Generic titles are: invoice, tax invoice, sales invoice, commercial invoice, proforma invoice, bill, receipt, cash memo, credit note, quotation, estimate.
3. **Clean-up.** A generic title at the end of the line is removed (`Example Supplies Ltd INVOICE` becomes `Example Supplies Ltd`). A stray OCR prefix such as `M1)` before a name is removed.
4. **Plausibility check.** At most 80 characters, does not start with a digit, has no `:` or `@`, no `http` or `www.`, at least two letters, and more letters than digits.
5. **Context check.** The vendor is accepted only if labelled invoice fields appear on later lines. If the first suitable line names another party or holds a field value itself, the result is `None`.

6.5 Missing-field behavior and fallback

- A field that is missing, malformed, or ambiguous is `None` in Python and `null` in JSON. The extractor never raises an error for bad text, only for non-string input.
- Lines with several fields are not split in the strict stage, so such a field returns `None` rather than a guess.
- The API does not fail when fields are `null`. The status is still `processed`.
- **Fallback processing** is described in section 3.2 and 5.2: pass 2 (PSM 4), then pass 3 (0.75x scale, PSM 4). Each pass runs only if at least one field is still `None` and fills only the `None` fields.
- Extraction stays the same in every pass. Only the OCR text changes.

6.6 Illustrative example of the fallback

The text below is **invented for illustration**. It is not real OCR output and not taken from a real invoice. It shows how the design is meant to work.

Pass 1 (PSM 6), cleaned text:

```
Example Supplies Ltd INVOICE
Invoice No: A-1001
Invoice Date: 12 March 2026
Item A 2 50.00 100.00
Tax 10.00 110.00
```

The invoice number, date and vendor are found. The subtotal, tax and total have no usable labelled line, so they stay `null`. Nothing is calculated from the other numbers.

Pass 2 (PSM 4), cleaned text:

```
Example Supplies Ltd INVOICE
Invoice No: A-1001
Invoice Date: 12 March 2026
Subtotal 100.00
Tax 10.00
Total 110.00
```

Pass 2 finds all six fields. Only the three fields that were `null` after pass 1 are taken from it. The final result is:

```
{"invoice_number": "A-1001", "invoice_date": "12 March 2026",
 "vendor_name": "Example Supplies Ltd", "subtotal": "100.00",
 "tax_amount": "10.00", "total_amount": "110.00"}
```

This shows three ideas of the design: no guessing, the fallback fills gaps only, and values found earlier are never replaced. In a real run, a fallback pass may or may not fill a field.

6.7 Known limits of the extraction logic

- Rules depend on recognizable labels. An invoice without the supported labels returns `null` fields.
- Corrupted labels are not repaired.
- Only the six fields are extracted. Line items are not.
- Dates are not converted to a standard format. They are returned as written.
- Extraction was designed around the labels and layouts the rules support. Other layouts may return `null` fields, and accuracy on other invoices has not been measured.

7. API Documentation

The API is developed with FastAPI and has three endpoints. The application title is "Intelligent Image Document Analyzer" and its version is `0.3.0`. There is no authentication.

Table 12: API endpoints

Method	Route	Purpose	Success status
`GET`	`/`	Identify the project and confirm the API is running	200
`GET`	`/health`	Health check	200
`POST`	`/upload`	Validate an invoice image, run OCR, extract six fields	200

FastAPI also provides `/docs` (Swagger UI), `/redoc` and `/openapi.json` automatically. These are not code written for this project.

7.1 `GET /`

- **Purpose:** shows the project name and that the API is running.
- **Parameters and body:** none.
- **Response (200):**

```
{
  "project": "Intelligent Image Document Analyzer",
  "message": "API is running"
}
```

- **Errors:** none defined by the project.

7.2 `GET /health`

- **Purpose:** lets a person or tool check that the server answers. It does not check Tesseract or any other component.
- **Parameters and body:** none.
- **Response (200):**

```
{
  "status": "healthy"
}
```

7.3 `POST /upload`

- **Purpose:** processes one invoice image and returns image details, the OCR text and the six extracted fields.
- **Request format:** `multipart/form-data`.
- **Required parameter:**

Table 13: Request parameter of `POST /upload`

Name	Where	Type	Required	Description
`file`	form field	file	Yes	Invoice image. JPEG, PNG, WebP or TIFF. Maximum 10 MB and 40,000,000 pixels.

The image format is detected from the file contents, not from the filename or the `Content-Type` header. PDF files are not supported.

Example request (curl):

```
curl -X POST "http://127.0.0.1:8000/upload" \
  -F "file=@path/to/invoice.png"
```

Example response (200). The values are invented placeholders that show the shape. The text fields are shortened to `...`. A response produced by the application is shown in Figure 7.

```
{
  "filename": "example_invoice.png",
  "format": "PNG",
  "width": 1000,
  "height": 1400,
  "size_bytes": 123456,
  "status": "processed",
  "message": "...",
  "raw_ocr_text": "...",
  "cleaned_text": "...",
  "invoice_fields": {
    "invoice_number": "INV-0001",
    "invoice_date": "2026-01-15",
    "vendor_name": "Example Supplies Ltd",
    "subtotal": "100.00",
    "tax_amount": "10.00",
    "total_amount": "110.00"
  }
}
```

Response fields

Table 14: Response fields of `POST /upload`

Field	Type	Meaning
`filename`	string	Filename sent by the client (`"unknown"` if none was sent).
`format`	string	Format detected by Pillow (`JPEG` , `PNG` , `WEBP` or `TIFF`).
`width` , `height`	integer	Image size in pixels.
`size_bytes`	integer	Size of the upload.
`status`	string	Always `"processed"` in a 200 response.
`message`	string	Human-readable summary.
`raw_ocr_text`	string	Text from the first OCR pass, unchanged.
`cleaned_text`	string	`raw_ocr_text` after whitespace cleaning.
`invoice_fields`	object	Six fields. All six keys are always present. A field not found is `null` . Amounts are decimal strings.

Status codes

Table 15: Status codes of `POST /upload`

Code	When	Example `detail`
200	Image validated and processed. Some fields may be `null` .	(response above)
400	Empty file, not a recognizable image, or corrupt/incomplete image	`Uploaded file is empty.`

Code	When	Example `detail`
413	Larger than 10 MB, or more than 40,000,000 pixels	`File is too large. Maximum allowed size is 10 MB.`
415	Valid image of another format, for example GIF	`Unsupported image format: GIF. Allowed formats: JPEG, PNG, WebP, TIFF.`
422	The `file` field is missing, or the image passed validation but OpenCV could not decode it	see Table 16
500	OCR failed (for example Tesseract is not installed)	`Text recognition failed on the server. Please try again later.`

Error responses

Table 16: Error responses and their messages

Case	Status	`detail`
Empty file	400	`Uploaded file is empty.`
Not a recognizable image	400	`File is not a recognizable image.`
Corrupt or incomplete image	400	`Image is corrupt or incomplete.`
Larger than 10 MB	413	`File is too large. Maximum allowed size is 10 MB.`
More than 40,000,000 pixels	413	`Image dimensions are too large.`
Valid image of another format (for example GIF)	415	`Unsupported image format: GIF. Allowed formats: JPEG, PNG, WebP, TIFF.`
Missing `file` field	422	FastAPI's standard validation error (a list).
Image passed validation but could not be decoded	422	`The image passed validation but could not be decoded for processing.`
OCR failure	500	`Text recognition failed on the server. Please try again later.`

Errors raised by the project use `{"detail": "text"}`. The missing-field error is produced by FastAPI and uses a list.

7.4 Using Swagger UI

Swagger UI is the interactive page FastAPI builds from the code. It lets you test the API without writing any code.

1. Start the server (section 10.6).
2. Open `http://127.0.0.1:8000/docs` in a browser.
3. Click `POST /upload`, then **Try it out**.
4. Click **Choose File** and select an invoice image.
5. Click **Execute**.
6. Read the **Server response** section: the status code and the JSON body appear there. The **Responses** table above it lists the codes documented in the code (200, 400, 413, 415, 422, 500).

The Swagger page can also test `GET /` and `GET /health` the same way. Screenshots of the interface and of a response are shown in section 7.5.

7.5 Application Demonstration and API Output

The screenshots in this section were taken from the running application's Swagger UI, the interactive page that FastAPI generates at `/docs`. They are reproduced exactly as provided and have not been edited. They illustrate the interface and the structure of one response. They do not show extraction accuracy, performance or production readiness.

Swagger UI interface. Figure 5 shows the Swagger UI page of the application. It displays the application title, the version badge `0.3.0`, the OpenAPI 3.1 badge, a link to `/openapi.json`, and the three operations: `GET /` (Read Root), `GET /health` (Health Check) and `POST /upload` (Upload Invoice Image). Swagger UI lets a person try each endpoint in the

browser without writing client code.

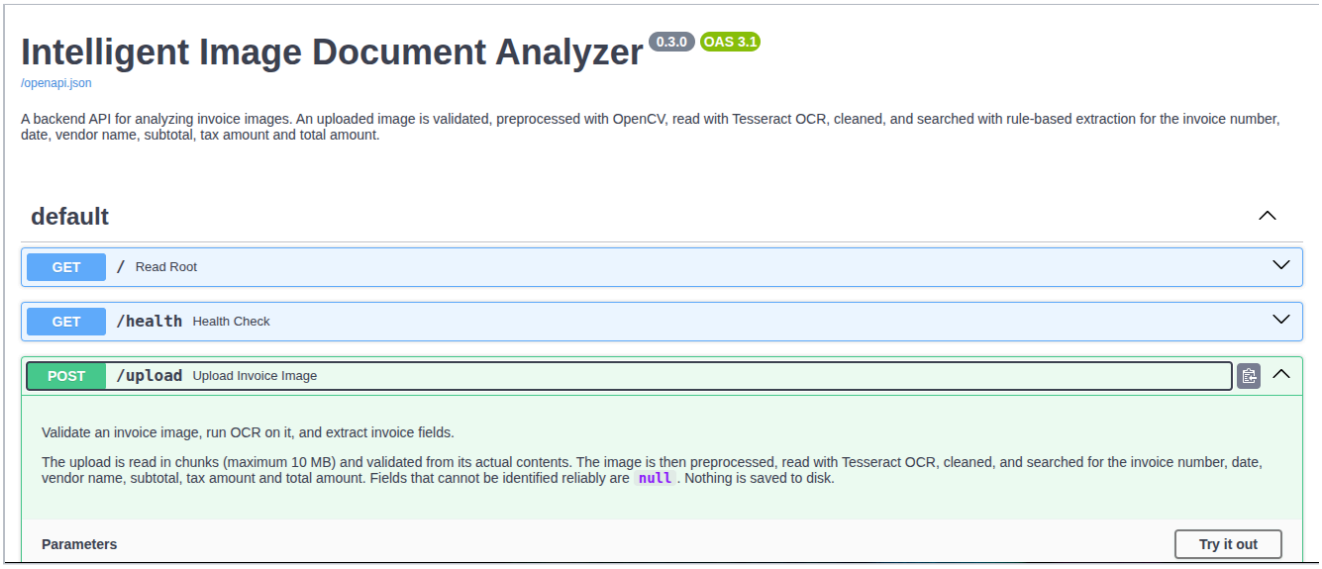


Figure 5: Swagger UI of the Intelligent Image Document Analyzer, showing the API version and the endpoints `GET /`, `GET /health` and `POST /upload`.

Invoice image upload. Figure 6 shows the expanded `POST /upload` operation in **Try it out** mode. The request body is `multipart/form-data` with one required form field, `file`, described as "Invoice image (JPEG, PNG, WebP, TIFF)". An invoice image is chosen with the **Choose File** button and submitted with **Execute**. The operation description visible in the screenshot states that the upload is read in chunks (maximum 10 MB), validated from its actual contents, preprocessed, read with Tesseract OCR, cleaned, and searched for the six invoice fields; that fields which cannot be identified reliably are `null`; and that nothing is saved to disk. The screenshot shows the form before a file is chosen.

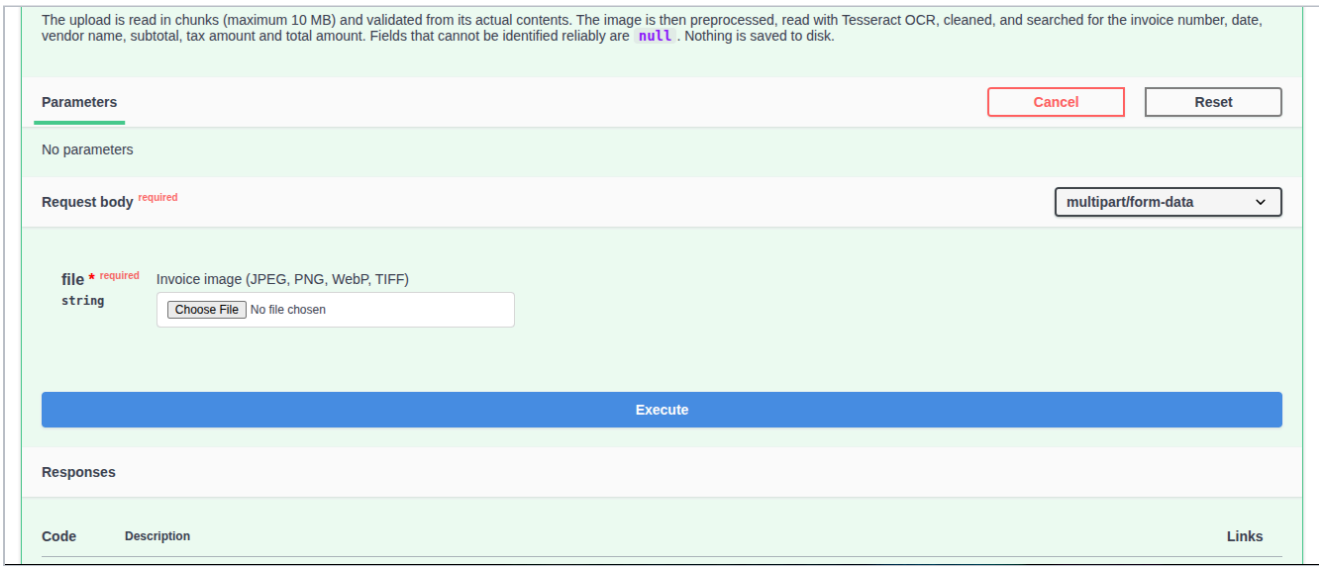


Figure 6: The `POST /upload` form in Swagger UI, with the required `file` field and the Execute button.

Successful JSON response. Figure 7 shows a response returned by the application in Swagger UI. The status code is 200, which means the uploaded image passed validation and was processed. The response body is structured JSON. It contains the image metadata (`filename`, `format`, `width`, `height`, `size_bytes`), the `status` value `processed`, a `message`, the OCR text of the first pass (`raw_ocr_text`), the cleaned form of that text (`cleaned_text`) and the `invoice_fields` object. The response headers show the content type `application/json`. The `raw_ocr_text` and `cleaned_text` values visible in the screenshot show that OCR output can contain recognition noise, which is why the extraction rules return `null` instead of guessing.

200

Response body


```
{
  "filename": "invoice_test_01.png",
  "format": "PNG",
  "width": 1700,
  "height": 2200,
  "size_bytes": 146617,
  "status": "processed",
  "message": "Image was validated and processed successfully.",
  "raw_ocr_text": "Northstar Office Supplies INVOICE\n\nPlot 21, Sector I-9 Industrial Area | ice No: INV-2026-00125\n\nIslamabad 44790, Pakistan noice 0: : .\n\nTel: +92 51 555 0142 | NTN: 1234567-8 Invoice Date: 24 October 2026\nTerms: Net 30 days\n\nBILL TO\n\nZenith Educational Trust\n\nHouse 7, Street 15, F-8/3\n\nIslamabad, Pakistan\n\ncr)\n\nA4 Copier Paper {ream} 40 650.00 26,000.00\n\nBallpoint Pens {box of 50) 480.00 12,000.00\n\nHeavy Duty Stapler 1,250.00 12,500.00\n\nfa Manila File Folders {pack} 120.00 6,000.00\n\nPayment Terms: Payment due within 30 days of the invoice date. 3\n\nPlease quote the invoice number with your payment. GST (18%) 10,170.00\n\nBank: Example Commercial Bank | A/C No: 0011 2233 4455 66,670.00\n\nThank you for your business.\n",
  "cleaned_text": "Northstar Office Supplies INVOICE\n\nPlot 21, Sector I-9 Industrial Area | ice No: INV-2026-00125\n\nIslamabad 44790, Pakistan noice 0: : .\n\nTel: +92 51 555 0142 | NTN: 1234567-8 Invoice Date: 24 October 2026\nTerms: Net 30 days\n\nBILL TO\n\nZenith Educational Trust\n\nHouse 7, Street 15, F-8/3\n\nIslamabad, Pakistan\n\ncr)\n\nA4 Copier Paper {ream} 40 650.00 26,000.00\n\nBallpoint Pens {box of 50) 480.00 12,000.00\n\nHeavy Duty Stapler 1,250.00 12,500.00\n\nfa Manila File Folders {pack} 120.00 6,000.00\n\nPayment Terms: Payment due within 30 days of the invoice date. 3\n\nPlease quote the invoice number with your payment. GST (18%) 10,170.00\n\nBank: Example Commercial Bank | A/C No: 0011 2233 4455 66,670.00\n\nThank you for your business.",
  "invoice_fields": {
    "invoice_number": "INV-2026-00125",
    "invoice_date": "24 October 2026",
    "vendor_name": "Northstar Office Supplies",
    "subtotal": "56500.00",
    "tax_amount": "10170.00",
    "total_amount": "66670.00"
  }
}
```


Download

Response headers

```
content-length: 1921
content-type: application/json
date: Tue,03 Nov 2026 16:58:18 GMT
server: uvicorn
```

Responses

Figure 7: A successful `POST /upload` response (HTTP 200) shown in Swagger UI.

Extracted invoice information. The `invoice_fields` object at the end of the response in Figure 7 holds the six supported fields. The information is already visible in Figure 7, so no separate figure is used. The fields are:

- `invoice_number`: the invoice number, as written in the OCR text.
- `invoice_date`: the invoice date, as written in the OCR text.
- `vendor_name`: the name of the vendor (the company that issued the invoice).
- `subtotal`: the amount before tax, as a decimal string.
- `tax_amount`: the tax amount, as a decimal string.
- `total_amount`: the total amount, as a decimal string.

A field that cannot be identified reliably is `null`. The screenshot shows the output for one uploaded image. It does not by itself show how accurate the extraction is on other invoices, and the extracted values should be reviewed by a person before financial use.

8. Automated Testing

8.1 Types of tests used in this project

Table 17: Types of tests used in the project

Type	Meaning	Where it is used
Unit test	Tests one function or module alone, with simple inputs.	`test_image_processing`, `test_text_cleaning`, `test_ocr`, `test_extraction`
API test	Sends HTTP requests to the application in memory and checks status codes and JSON.	`test_api`
Integration test	Tests several modules together. Slow or unavailable parts (Tesseract) are replaced by fakes.	`test_pipeline`, `test_invoice_regression`
Regression test	Keeps real examples that once failed, so a later change cannot break them silently.	`test_invoice_regression`, and the real-OCR sections of `test_extraction`
Invalid-input test	Gives wrong types, empty data, bad options or corrupt files and checks that the correct error is raised.	In every file
Edge-case test	Tests unusual but possible input: Unicode text, tabs, leading zeros, alpha channels, labels with no value.	In every file
OCR-related test	Checks OCR settings, error mapping and fallback behavior. Tesseract is mocked in all but one test.	`test_ocr`, `test_pipeline`, `test_invoice_regression`

Important: almost all tests run without Tesseract. The regression tests use text that Tesseract produced earlier and that was saved in `tests/invoice_ocr_samples.py` and `tests/invoice_regression_cases.py`. They test the extraction rules and the fallback logic, **not** Tesseract itself. Only the opt-in test `test_real_tesseract_reads_simple_rendered_text` starts the real engine.

8.2 Test files

The project has seven test files and two test-data modules in `tests/`. Passing tests show that the code behaves as the tests specify. They do not prove accuracy on real-world invoices.

Table 18: Test files, purpose and important scenarios

Test file	Purpose	Important scenarios tested
`tests/test_api.py`	API behavior with FastAPI's `TestClient`, using in-memory images and a stand-in for the pipeline.	Root and health responses. Valid PNG and JPEG upload. All six fields present, with `null` for missing ones. Uploaded bytes are passed to the pipeline. Empty file (400). Text file named `.png` (400). GIF (415). Over 10 MB (413). Over the pixel limit (413). Missing file (422). Decode error (422). OCR error (500 with a safe message).
`tests/test_image_processing.py`	Preprocessing functions with synthetic NumPy images.	Output is 2-D with only 0 and 255. Grayscale, 3-channel, 4-channel and 1-channel input. Input array is not modified. Gaussian and median blur. Otsu and adaptive thresholds. Invalid arrays and invalid options raise the documented errors.
`tests/test_text_cleaning.py`	`clean_ocr_text` rules.	Line-ending styles. Whitespace reduction and line trimming. Blank-line reduction. Non-string input raises `InvalidTextInputError`.
`tests/test_ocr.py`	`app/ocr.py` with Tesseract mocked.	Image input types. Language, PSM and extra config handling. Missing Tesseract (`OCREngineNotFoundError`). Other engine failures. One opt-in test uses real Tesseract (see below).
`tests/test_extraction.py`	Extraction rules (chapter 6).	Label spelling, case and spacing. Money formats. Subtotal, tax and total kept apart. Value on the next line. Vendor rules. Table-style layouts. Recorded OCR text of sample invoices. Garbled text returns all `None`. A corrupted label is not repaired.
`tests/test_pipeline.py`	`decode_image` and `process_invoice_image` with recording fakes.	Decoding of supported formats. Invalid bytes. Steps run in the documented order. Fallback is off by default, runs only when a field is missing, and fills only `None` fields. Fallback errors are logged and the primary result is kept. Invalid PSM or scale is rejected before any OCR.
`tests/test_invoice_regression.py`	Regression cases (`Case` records) run through the pipeline with scripted OCR text.	Expected fields are returned with fallback and rescale. A pass returns the right value or `None`. Fallback never overwrites a value found earlier.
`tests/invoice_ocr_samples.py`	Test data only: recorded OCR text and expected fields.	Used by other test files.
`tests/invoice_regression_cases.py`	Test data only: `Case` records.	Used by `test_invoice_regression`.

The opt-in real-OCR test. `test_real_tesseract_reads_simple_rendered_text` in `tests/test_ocr.py` draws text with OpenCV and reads it with real Tesseract. It is skipped unless the environment variable `RUN_OCR_INTEGRATION=1` is set and `tesseract` is installed.

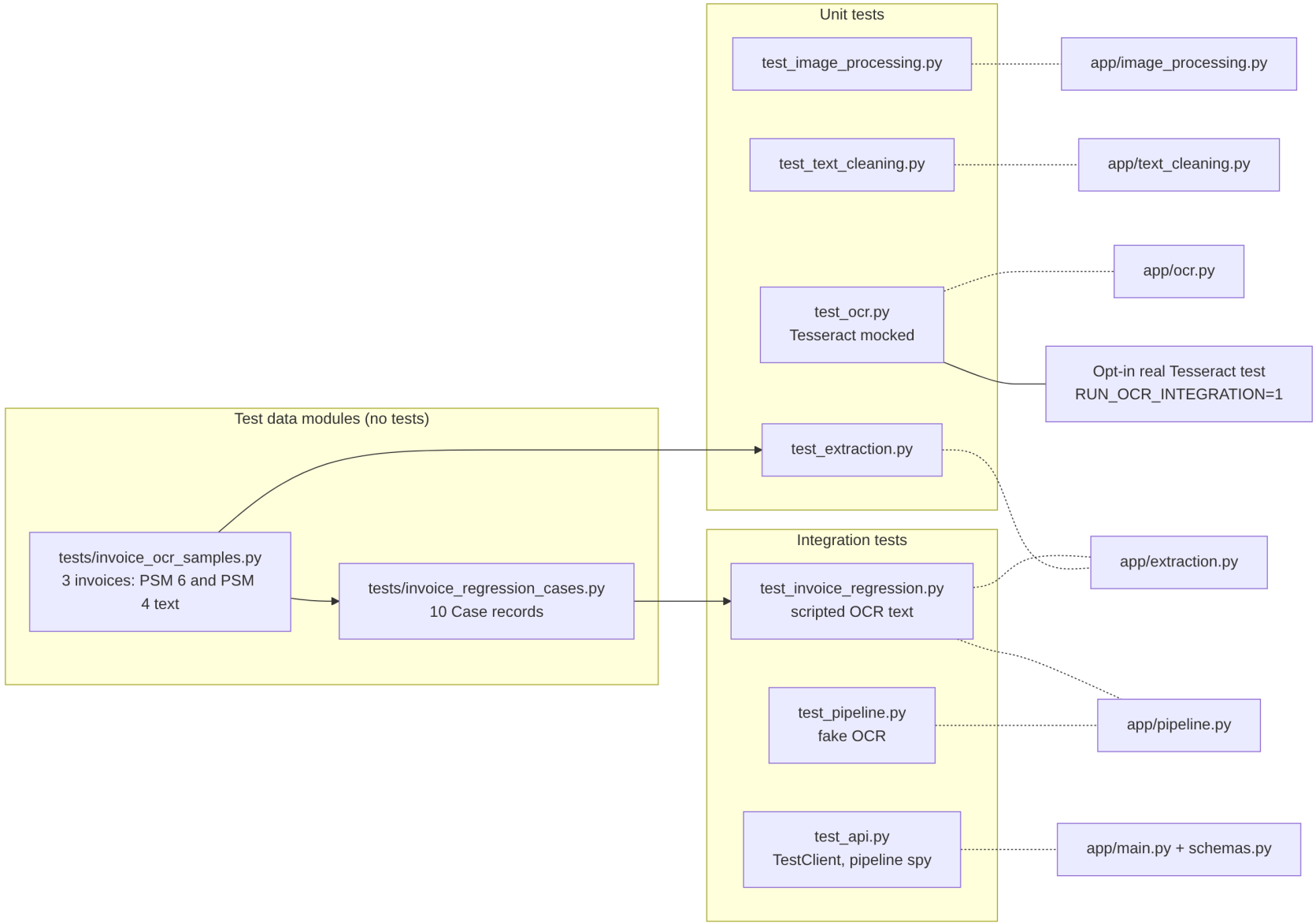


Figure 8: How the test files relate to the modules and the test data. Dotted lines connect each test file to the module it tests. Source: `diagrams/05-testing-structure.mmd`.

8.3 Commands

Run from the project root with the virtual environment active (see section 10).

```
# All tests
python -m pytest

# All tests, one line per test
python -m pytest -v

# One test file
python -m pytest tests/test_api.py -v
python -m pytest tests/test_extraction.py -v
python -m pytest tests/test_image_processing.py -v
python -m pytest tests/test_invoice_regression.py -v
python -m pytest tests/test_ocr.py -v
python -m pytest tests/test_pipeline.py -v
python -m pytest tests/test_text_cleaning.py -v

# One test by name
python -m pytest tests/test_pipeline.py -k "fallback" -v

# Include the real-Tesseract test (Linux or macOS)
RUN_OCR_INTEGRATION=1 python -m pytest tests/test_ocr.py -v
```

The project has no `pytest.ini` or `pyproject.toml`; the tests import `invoice_ocr_samples` by name, which works when pytest is started from the project root.

8.4 Test results

This report does not state test results. Run `python -m pytest` in your own environment to see them. Results depend on the installed Python, packages and Tesseract.

Coverage. Code coverage was not measured, so no coverage percentage is given.

What the tests do not show. Passing tests do not prove real-world accuracy. The OCR text in most tests is recorded or faked, so they do not measure OCR accuracy on new images. They do not test performance or concurrent use.

9. OCR Evaluation Experiments

9.1 What `scripts/evaluate_ocr.py` is

A command-line script for comparing OCR settings on **one image** by eye. It uses the project's own functions (`preprocess_image`, `to_grayscale`, `reduce_noise`, `extract_text`, `clean_ocr_text`), so the "current defaults" run uses the same preprocessing as the API's first pass.

It prints results only. It does not compare against correct answers, does not give a score, does not rank settings, and does not write any file or change any module. It reads the image with `cv2.imread` directly from disk, not through the API.

9.2 What it varies

Experiment 1: blur and page segmentation (6 runs)

Table 19: Variations in Experiment 1

Preprocessing	PSM values
Current defaults (`preprocess_image(image)`: grayscale, Gaussian blur kernel 3, Otsu threshold)	4, 6, 11
No blur (`preprocess_image(image, kernel_size=1)`)	4, 6, 11

Experiment 2: scale and thresholding (12 runs, no blur in any run)

Table 20: Variations in Experiment 2

Option	Values
Scale factor	1x (original pixels), 2x, 3x. Resized with `cv2.resize` and `INTER_CUBIC`.
Preprocessing	"Grayscale, no threshold" (`to_grayscale` then `reduce_noise` with kernel size 1) and "Otsu threshold" (`preprocess_image` with kernel size 1 and Otsu)
PSM	4, 6

PSM means Tesseract's page segmentation mode. In short: 4 reads text as columns of different sizes, 6 reads one uniform block of text, 11 looks for sparse text in any order. The project's API uses PSM 6 first and PSM 4 as the fallback. Adaptive thresholding exists in `app/image_processing.py` but is not part of these experiments.

9.3 How to run it

```
# Show the options
python scripts/evaluate_ocr.py --help

# Run both experiments
python scripts/evaluate_ocr.py path/to/invoice.png

# Run one experiment
python scripts/evaluate_ocr.py path/to/invoice.png --experiment 1
python scripts/evaluate_ocr.py path/to/invoice.png --experiment 2
```

Larger scale factors make the image larger, so Experiment 2 may take longer on big images.

Script errors. The script exits with a message if the path does not exist, is not a file, OpenCV cannot read it, a required function option is missing, or Tesseract is not installed. If Tesseract fails for one configuration only, that run prints `OCR failed for this configuration` and the others continue.

9.4 How to read the output

Each run prints a block separated by a line of `=` signs. The layout is:

```
Configuration: preprocessing: current defaults | psm 4
Image passed to OCR: <width> x <height> pixels
-- Raw OCR output (repr, so whitespace is visible):
'<raw text, with \n shown as characters>'
-- Cleaned OCR output:
<cleaned text>
```

To interpret it, compare the blocks by eye:

- Are the invoice number, date and amounts present and complete?
- Are labels such as `Invoice No` and `Total` intact? The extraction rules need them.
- Are the digits, commas and decimal points correct?
- Does the order of lines still make sense?

Record your own notes. A setting that looks better on one image may not be better on another.

9.5 Experiments, tests and benchmarks are different things

Table 21: Manual experiment, automated test and formal benchmark compared

	Manual OCR experiment (`evaluate_ocr.py`)	Automated test (`pytest`)	Formal accuracy benchmark
Question answered	"What does Tesseract output with these settings?"	"Does the code still behave as specified?"	"How often is the system correct on a labelled dataset?"

	Manual OCR experiment (`evaluate_ocr.py`)	Automated test (`pytest`)	Formal accuracy benchmark
Judgement made by	A person, by eye	Assertions in code, pass or fail	A scoring script against ground-truth values
Needs Tesseract	Yes	Almost never	Yes
Exists in this project	**Yes**	**Yes**	**No**

No experimental findings are given in this document. The script prints text only, and no experiment results or scores are included here. Any observation made by eye on one image is not a measured result.

10. Installation and Execution Guide

These steps are for Ubuntu Linux. Other operating systems are not covered.

10.1 Prerequisites

You need:

- **Python 3** with `venv` and `pip`. The repository does not state a required version, and the lowest working version was not determined.
- **Git**.
- **Tesseract OCR** (section 10.4).

Check what you have:

```
python3 --version
git --version
```

On Ubuntu, if `venv` is missing: `sudo apt install python3-venv python3-pip`.

10.2 Clone the repository

```
git clone https://github.com/jawadali-s/intelligent-document-analyzer.git
cd intelligent-document-analyzer
```

10.3 Create the environment and install dependencies

A virtual environment keeps this project's packages separate from the rest of your computer.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Your terminal prompt should now start with `(.venv)`. On Windows (PowerShell) the activation command is `.venv\Scripts\Activate.ps1`. Run every later command in the same terminal with the environment active.

10.4 Install and verify Tesseract OCR

`pip` does not install Tesseract. On Ubuntu:

```
sudo apt update
sudo apt install tesseract-ocr
```

Verify:

```
tesseract --version
```

You should see a version line. The English language data (`eng` , the default language) is needed. If you see "command not found", the API will answer `/upload` requests with status 500. Other operating systems are not covered here.

10.5 Optional quick check

```
python -m pytest -q
```

The tests should run and report pass or fail counts for your environment.

10.6 Start the server

```
python -m uvicorn app.main:app --reload
```

The server listens at `http://127.0.0.1:8000` . `--reload` restarts it when code changes and is meant for development. Stop it with `Ctrl + C` .

10.7 Open Swagger UI and upload an invoice

1. Open `http://127.0.0.1:8000/docs` in a browser.
2. Use `POST /upload` as described in section 7.4.

Or use the command line, in a second terminal:

```
curl -X POST "http://127.0.0.1:8000/upload" \
  -F "file=@path/to/invoice.png"
```

OCR runs on the image, so a request can take some time. Run time was not measured.

10.8 Run the tests

See section 8.3. The usual command is:

```
python -m pytest -v
```

10.9 Run OCR evaluation experiments

See section 9.3:

```
python scripts/evaluate_ocr.py path/to/invoice.png --experiment 1
```

10.10 Common problems

Table 22: Common problems and fixes

Problem	Likely cause and fix
`tesseract: command not found` , or `/upload` returns 500	Tesseract is not installed or not on the `PATH` . Install it (section 10.4). The server log shows the detail; the response does not.
`ModuleNotFoundError: No module named 'fastapi'`	The virtual environment is not active, or the dependencies were not installed. Run the activation and `pip install -r requirements.txt` again.
`ModuleNotFoundError: No module named 'app'`	Run commands from the project root, the folder that contains `app/` .

Problem	Likely cause and fix
`Address already in use`	Another program uses port 8000. Use `--port 8001` in the `uvicorn` command.
415 for a photo	The file is not JPEG, PNG, WebP or TIFF, for example HEIC or PDF. Convert it first.

11. Input Validation and Error Handling

11.1 Safeguards implemented

Figure 9 shows the order in which an upload is checked and the status code that each failure gives. The first failing check ends the request.

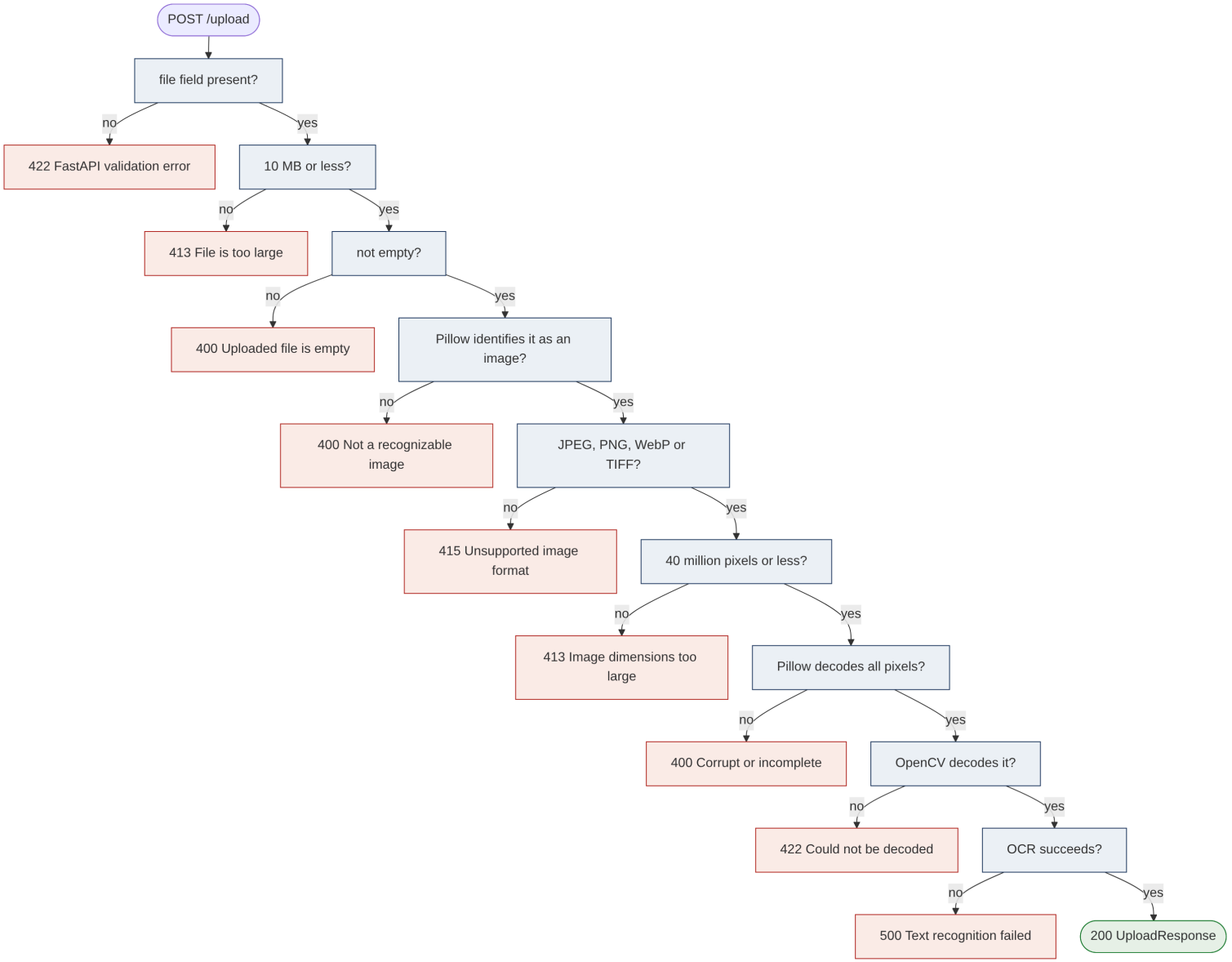


Figure 9: Order of the upload checks and the HTTP status returned for each failure. Source: `diagrams/06-error-handling.mmd`. The "Pillow" checks are in `validate_image` in `app/main.py`.

Table 23: Input validation and error handling safeguards

Safeguard	Where	How it works	Result
File size limit	`main.py`	10 MB. If the client declares a size above it, the upload is refused before reading. Otherwise it is read in 1 MB chunks and stops as soon as the total passes 10 MB.	413
Empty file	`main.py`	Zero bytes after reading.	400
Supported formats	`main.py`	Pillow detects the real format. Only JPEG, PNG, WebP and TIFF are accepted, regardless of filename or `Content-Type`.	415
Image dimension limit	`main.py`	Width × height above 40,000,000 pixels. Pillow's own decompression-bomb limit is set to the same value, and its warning is treated as an error inside the validation function only.	413
Invalid image	`main.py`	Pillow cannot identify the file as an image.	400
Corrupt or truncated image	`main.py`	Two passes: `verify()`, then reopen and `load()` to decode all pixels. Pillow errors (`OSError`, `SyntaxError`, `ValueError`, `EOFError`) become 400.	400
Decode failure after validation	`pipeline.py`, `main.py`	OpenCV cannot decode what Pillow accepted. `ImageDecodeError` becomes 422 with a fixed message.	422
OCR failure	`ocr.py`, `main.py`	All Tesseract problems become an `OCRError` type. The API returns a fixed message and logs the details on the server.	500
Extra OCR pass failure	`pipeline.py`	An `OCRError` in pass 2 or 3 is logged as a warning. The earlier result is returned and the request still succeeds.	200
Input type checks	each module	Wrong types raise specific exceptions: `InvalidImageBytesInputError`, `InvalidImageError`, `InvalidParameterError`, `OCRInputError`, `OCRConfigError`, `InvalidTextInputError`, `InvalidExtractionInputError`.	Internal
Bad pipeline options	`pipeline.py`	An unsupported `fallback_psm`, a bad `rescue_scale`, or a rescale without a fallback is rejected before any OCR runs.	Internal
No storage	`main.py`	The image is held in memory and released in a `finally` block. Nothing is written to disk by the endpoint.	n/a
Blocking work	`main.py`	Validation and the pipeline run in worker threads, so the event loop stays free.	n/a
Unexpected errors	`main.py`	Only known error types are converted. Any other exception is not hidden, so FastAPI returns its default 500 response.	500

11.2 Pipeline exceptions

Table 24: Pipeline exceptions and how the API handles them

Exception	Raised by	Handled in the API?
`ImageDecodeError`	`decode_image`	Yes, 422
`OCRError` and subclasses	`ocr.extract_text`	Yes, 500
`ImagePreprocessingError`	`image_processing`	**No.** It is not caught in `main.py`.
`InvalidImageBytesInputError`, `InvalidTextInputError`, `InvalidExtractionInputError`	pipeline modules	No. They signal a programming error, not a bad upload.

11.3 Purpose and limits of these controls

The controls make the API refuse input that is the wrong kind, too big or damaged, and they keep internal OCR error details out of error responses. They are input checks, not a complete security solution:

- The project has no authentication, user accounts, rate limiting, database or permanent storage. Uploaded images are held in memory only.
- The 10 MB limit applies to each request. The number of simultaneous requests is not limited.
- Validation checks that a file is a valid image. It does not check that it is an invoice.
- The application is not documented as ready for public deployment.

12. Results and Limitations

12.1 Implemented capabilities

Table 25: Implemented capabilities and the tests that cover them

Capability	Where it is covered
Three-endpoint FastAPI app with documented responses and status codes	`app/main.py`, `tests/test_api.py`
Upload validation: size, format, pixel count, empty, corrupt and non-image files	`app/main.py`, `tests/test_api.py`
OpenCV preprocessing and Tesseract OCR with typed errors	`tests/test_image_processing.py`, `tests/test_ocr.py`
Text cleaning of line endings and whitespace	`tests/test_text_cleaning.py`
Rule-based extraction of six fields, with `null` when unsure	`tests/test_extraction.py`
Fallback passes that only fill empty fields	`tests/test_pipeline.py`, `tests/test_invoice_regression.py`

The quality of the results on real invoices is **unknown**. The project has no accuracy measurement.

12.2 Limitations

Table 26: Known limitations

Area	Limitation
OCR accuracy	Extraction can only be as good as the OCR text. Tesseract may drop labels or misread digits and punctuation.
Image quality	Small, blurry, skewed or low-resolution images read poorly.
Layout variation	The rules depend on common English labels at line starts or in table cells. Unusual layouts, other languages, handwriting and multi-column pages may fail.
Rule-based extraction	Rules were developed from a small set of sample invoices. New layouts may need new rules, and every rule can wrongly match text in some case not tested.
Missing fields	A missing field is `null`, and the client must handle it. The API gives no per-field confidence or reason.
No guessing by design	Corrupted labels are not repaired and a missing amount is never calculated from the others. This avoids wrong values but leaves more `null` fields.
Values returned as written	Dates are not converted to one format.
Scope	Only six header fields. No line items, currency field, customer, PDF input or other languages.
Evaluation	No formal OCR accuracy evaluation has been performed. No labelled dataset is used, and there is no scoring script or accuracy figure.
Human review	Extracted values should be reviewed by a person before financial use.
Environment	Other Tesseract versions may give different OCR text.
Operation	No authentication, rate limiting, database or permanent document storage. No Docker setup, CI pipeline or deployment files.
Dependencies	`requirements.txt` is a full package listing without a stated Python version.

13. Future Improvements

The items below are **proposals**. None of them exists in the current code.

Table 27: Proposed improvements (not implemented)

Proposal	Why
Build a labelled invoice dataset and a scoring script (field-level correctness per field)	Makes accuracy measurable and lets changes be compared with numbers. This is the most important next step.
Use Tesseract word positions (bounding boxes) for layout-based extraction	Could handle label and value placed in different columns without line-based rules.
Add a per-field note on which pass or rule produced it	Helps users judge trust.
Add PDF input	Many invoices arrive as PDFs.
Add authentication, rate limits and a deployment guide	Needed before any public use.
Add continuous integration and a coverage report	The tests can run on every change.
Add a `pyproject.toml` or `pytest.ini` with the supported Python version	Makes setup and test runs more predictable.
Support more languages and more date and currency formats	Widens the range of invoices.

Their benefit is expected, not measured. Everything in chapters 1 to 12 describes functionality that exists today.

14. Conclusion

The Intelligent Image Document Analyzer reads an invoice image and returns the OCR text and six fields through a three-endpoint FastAPI service. The upload endpoint validates size, format, pixel count and file integrity, runs OpenCV preprocessing and Tesseract OCR, cleans the text, and extracts the fields with explicit rules. When a field is missing, up to two extra OCR passes fill only the empty fields. The code is divided into small modules with typed errors, and the API turns known failures into clear HTTP status codes.

The project includes an automated test suite that covers the API, each module and the fallback behavior, and an experiment script for manual comparison of OCR settings. What is missing is a measurement of accuracy on real invoices. Output should be checked by a person before it is used for financial purposes.

15. References and Appendices

15.1 References

Table 28: References

Topic	Source
Project source code, `requirements.txt` and tests	github.com/jawadali-s/intelligent-document-analyzer

The technologies named in this report (FastAPI, Uvicorn, Pydantic, Pillow, OpenCV, NumPy, Tesseract, pytesseract and pytest) are described in their own official documentation. Links to those pages are not listed, because they could not be opened and checked when this report was prepared.

15.2 Appendix A: Files generated for this documentation

These files were produced for this documentation. They are not part of the application, and this report does not confirm that they are in the repository.

Table 29: Documentation files

File	Description
`docs/Intelligent_Image_Document_Analyzer_Documentation.pdf`	This report as a PDF.
`docs/technical-report.md`	Markdown source of this report.
`docs/part1-architecture-and-implementation.md`	Chapters 1 to 6 as a separate file.
`docs/part2-api-testing-and-user-guide.md`	Chapters 7 to 15 as a separate file.
`docs/diagrams/`	Mermaid sources (`.mmd`) and PNG images for Figures 1 to 4, 8 and 9.
`docs/screenshots/`	Screenshots supplied by the author for Figures 5 to 7, used unedited.
`docs/build/`	Scripts and style sheet used to build the PDF.

15.3 Appendix B: Items not verified

Table 30: Items that could not be verified

Item	Status
Python and Tesseract versions on the author's computer	Not verified. No version is stated as confirmed.
Current test results	Not stated in this report.
Accuracy of extracted values on real invoices	Not available. No labelled dataset exists.
Installation on Windows or macOS	Not covered.
Lowest supported Python version	Not determined.
Run time of any step	Not measured.
Screenshots (Figures 5 to 7)	Supplied by the author. They show one response and are not a test of accuracy.
External documentation links	Not listed, because they could not be opened and checked.

15.4 Appendix C: Glossary

Table 31: Glossary

Term	Meaning
API	Application Programming Interface. Here: the HTTP endpoints of the service.
OCR	Optical Character Recognition. Converting the text in an image into computer text.
Tesseract	Open-source OCR engine used by the project.
PSM	Page segmentation mode. A Tesseract setting that says how to divide a page into text blocks. The project uses 6 first and 4 as the fallback.
Fallback pass	An extra OCR run that is used only to fill fields that are still `null` .
`null`	JSON value for "no value". Used for a field that could not be found reliably.